Checkpoint 3.2 — Client Profile Schema

Amen Bank — Real-Time Anomaly Detection Platform

Internship Project Documentation

June 3, 2026

1 Purpose

This checkpoint defines the complete **client profile schema** — both the **hot profile** stored in Redis for real-time lookup by the Enrichment Service, and the **cold profile** stored in PostgreSQL as the source of truth from which the hot profile is recomputed nightly by the batch pipeline.

This builds on Checkpoint 3 (raw event schema) and Checkpoint 3.1 (pipeline dependencies, cold start, archetypes).

2 Design Philosophy

2.1 Hot / Cold Split

- **Hot profile (Redis)** — precomputed aggregate statistics and distributions, optimized for O(1) lookup during real-time enrichment. Same *categories* as cold, but condensed.
- **Cold profile (PostgreSQL)** — raw transaction records, full alert history, master client data, and time-series balance snapshots. The batch pipeline reads from here to recompute the hot profile.

2.2 Five Categories

Both stores organise client data into the same five categories, derived by reasoning about what information the model needs to score an incoming event:

1. **Personal Info** — identity and registration data
2. **Transaction Statistics** — aggregate metrics over rolling windows
3. **Behavioral Patterns** — where, when, how the client transacts
4. **Financial State** — current balance, debt, account portfolio
5. **Risk History** — past alerts and supervisor decisions

2.3 Multi-Window Pattern

Most statistics and distributions are computed across multiple time windows simultaneously:

- Transaction statistics: `24h, 7d, 30d, 90d`
- Behavioral distributions: `30d` (short-term) and `180d` (long-term baseline)
- Financial state: `30d, 90d`

- Risk history: `24h, 7d, 30d, 90d`

Multiple windows let the model catch burst behavior (24h) versus gradual drift (90d+) simultaneously.

3 Hot Profile (Redis)

3.1 Category 1 — Personal Info (7 fields)

```
client_id              : string  (primary key)
archetype              : enum    [student, salaried, small_biz,
                                  retiree, big_biz]
maturity_status        : enum    [cold_start, mature]
home_branch_id         : string
client_type            : enum    [individual, business]
nationality            : string
account_opening_date   : date
```

3.2 Category 2 — Transaction Statistics (113 fields)

```
// Client-level
days_since_last_transaction : int

// Per operation type x per window
// operation_type in {retrait, versement, virement, cheque}
// window         in {24h, 7d, 30d, 90d}
stats_<operation_type>_<window> {
    count                : int
    sum_amount           : float
    mean_amount          : float
    std_amount           : float
    rejected_count       : int
    reversed_count       : int
    near_threshold_ratio : float   // proportion in 8,000-9,999 DT band
}
// 4 ops x 4 windows x 7 fields = 112 fields
// + 1 client-level field = 113 total
```

Notes:

- `near_threshold_ratio` targets smurfing detection (deposits deliberately kept just below the 10 000 DT LAB/FT reporting threshold).
- `std_amount` enables z-score computation: `(current - mean) / std`.
- `rejected_count` and `reversed_count` capture operational anomalies (potential employee error or coercion).

3.3 Category 3 — Behavioral Patterns (10 distributions)

Each behavioral dimension is stored as a frequency distribution (Python dict / Redis hash) at two time horizons.

```
behavior_30d {
    branch_distribution    : {branch_id      -> frequency}
    hour_distribution      : {hour_bucket    -> frequency}  // 8h-16h30
```

```
    dow_distribution       : {day_of_week    -> frequency}  // Mon-Fri
    operation_distribution : {operation_type -> frequency}
    employee_distribution  : {employee_id    -> frequency}
}

behavior_180d {
    // same 5 distributions as above
}
```

Why distributions, not single values: a client may have one dominant branch but also occasional secondary ones. A frequency distribution lets the model compute "how rare is this branch for this client" rather than a binary match/mismatch.

Why two horizons: short-term (30d) captures recent habits; long-term (180d) captures true baseline. Divergence between the two is itself a signal that behavior is shifting.

Scope note: Hour and day-of-week distributions are bounded by teller working hours (Monday–Friday, 08:00–16:30). This may extend if the supervisor approves adding GAB (ATM) operations.

3.4 Category 4 — Financial State (12 fields)

```
current_balance              : float
current_debt                 : float
num_accounts                 : int
account_types_distribution   : {account_type -> count}

avg_balance_30d              : float
std_balance_30d              : float
avg_balance_90d              : float
std_balance_90d              : float

total_inflow_30d             : float
total_outflow_30d            : float
total_inflow_90d             : float
total_outflow_90d            : float
```

Why inflow/outflow separately: a client whose balance stays steady but whose monthly inflow jumps from 3 000 DT to 80 000 DT is suspicious even if the balance is unchanged — money flowed through.

3.5 Category 5 — Risk History (49 fields)

```
days_since_last_alert        : int

// Per tier x per window
// tier   in {info, review, alert, block}
// window in {24h, 7d, 30d, 90d}
count_<tier>_<window>            : int
confirmed_count_<tier>_<window>  : int
rejected_count_<tier>_<window>   : int

// 4 tiers x 4 windows x 3 fields = 48 fields
// + 1 client-level field = 49 total
```

Why split confirmed vs rejected: the supervisor feedback loop distinguishes true positives from false alarms. A client with 5 alerts last month, 4 confirmed, is genuinely high-risk. A client with 5 alerts, 4 rejected, has unusual-but-legitimate behavior — the model should be less trigger-happy.

3.6 Hot Profile — Field Count Summary

Category	Fields / structures
1. Personal info	7
2. Transaction statistics	113
3. Behavioral patterns	10 distributions
4. Financial state	12
5. Risk history	49
Total	**~191 per client**

4 Cold Profile (PostgreSQL)

The cold store holds the raw data from which the batch pipeline recomputes the hot profile each night. It is organised into several normalized tables.

```
clients_master {
    client_id (PK), archetype, account_opening_date,
    nationality, client_type, home_branch_id, kyc_data, ...
}

accounts {
    account_id (PK), client_id (FK), account_type,
    current_balance, current_debt, opening_date, status
}

account_balances {
    // Time series for trajectory analysis (avg_balance_*, std_*, etc.)
    account_id (FK), snapshot_date, balance, debt
}

transactions {
    // Every raw event ever persisted
    event_id (PK), client_id (FK), account_id (FK),
    employee_id (FK), branch_id, timestamp, amount,
    currency, operation_type, payload (JSON), status
}

alerts {
    // Every decision emitted by the Decision Service +
    // supervisor feedback from the Compliance Dashboard
    alert_id (PK), event_id (FK), client_id (FK),
    employee_id (FK), timestamp, severity_tier,
    anomaly_score, shap_explanation,
    supervisor_decision, supervisor_notes,
    decision_timestamp
}

profile_snapshots {  // optional
    // Historical hot-profile JSON for LSTM training and audit
    client_id (FK), snapshot_date, profile_json
}
```

4.1 Mapping — Hot Field → Cold Source

Hot Category	Cold Source Table(s)	Computation
Personal info	`clients_master`	direct copy
Transaction statistics	`transactions`	rolling-window aggregation
Behavioral patterns	`transactions`	rolling-window frequency tables
Financial state	`accounts`, `account_balances`	direct + rolling stats
Risk history	`alerts`	rolling-window counts split by decision

5 Open Questions

1. **Balance source.** Are daily balance snapshots provided by the core banking system, or do we receive balance-update events on a topic? This affects how `account_balances` is populated.

2. **Cold-start thresholds.** Exact values for `maturity_status` graduation (min transaction count and/or min time window) — to be calibrated during evaluation.

3. **Profile snapshot cadence.** Is the optional `profile_snapshots` table populated nightly, weekly, or only on-demand for LSTM training?

4. **Distribution truncation.** Behavioral distributions (branch, employee, etc.) can grow unbounded over 180 days. Should rare keys (frequency below a threshold) be folded into an "other" bucket to bound the size?

6 Next Step

Design the **employee profile schema** following the same hot/cold methodology, then define the **enriched event schema** produced by the Enrichment Service after joining a raw event with both client and employee profiles.